AI SMART WEIGHING POS

Camera-based produce recognition + Bluetooth weighing + automatic cart entry

Training dataset starter pack and implementation guide | Prepared for Yadav VeggiFruit Store

76	336	7
fresh produce classes	seed images + orientation variants	optional packaged SKUs



THE TARGET WORKFLOW

1. Place one product / one SKU on the weighing tray.
2. Bluetooth reports stable weight in grams.
3. The fixed overhead camera captures the scale area.
4. The vision model predicts the product class.
5. POS combines SKU + weight + price and adds one cart line.

IMPORTANT DATASET NOTE

The ZIP is a starter set built from existing catalog images. Each class has one seed image plus 3 synthetic rotations. These are not 4 independent camera-angle photographs. Use the capture plan in this guide to collect real scale-camera images before production training.

200 ms is a design target for model inference, not a guaranteed end-to-end response time. Benchmark the camera, Bluetooth, model, and cart pipeline together on the actual POS hardware.

Reference photo supplied by the store: current weighing scale and green perforated basket. Use this to plan the camera ROI; do not treat the whole photo as a labeled banana training image.

1. System design and camera placement

Recommended setup for the photo you supplied

• Camera: fixed overhead position, lens close to perpendicular to the tray. Frame the complete weighing surface with a small border; keep the scale display outside the recognition crop if it creates glare or visual noise.

• Distance: choose mounting height by field of view, not by a fixed guessed distance. The tray should fill most of the frame while remaining completely visible at every placement.

• Tray: either keep the same green perforated basket in every training and live image, or replace it with one shallow, matte, neutral tray. Do not alternate trays casually. A flat, non-patterned tray makes segmentation easier.

• Lighting: add diffuse LED lighting; lock exposure, focus, and white balance. Avoid sharp shadows, window glare, reflections, and flicker.

• Scale: tare/zero the empty tray once. The Bluetooth reading should represent net product grams, not tray + product.

The most important domain match is the real camera view. White-background catalog images are useful for a seed set, but the model must also see the actual tray, counter, lighting, and camera angle used at the shop.

End-to-end event flow

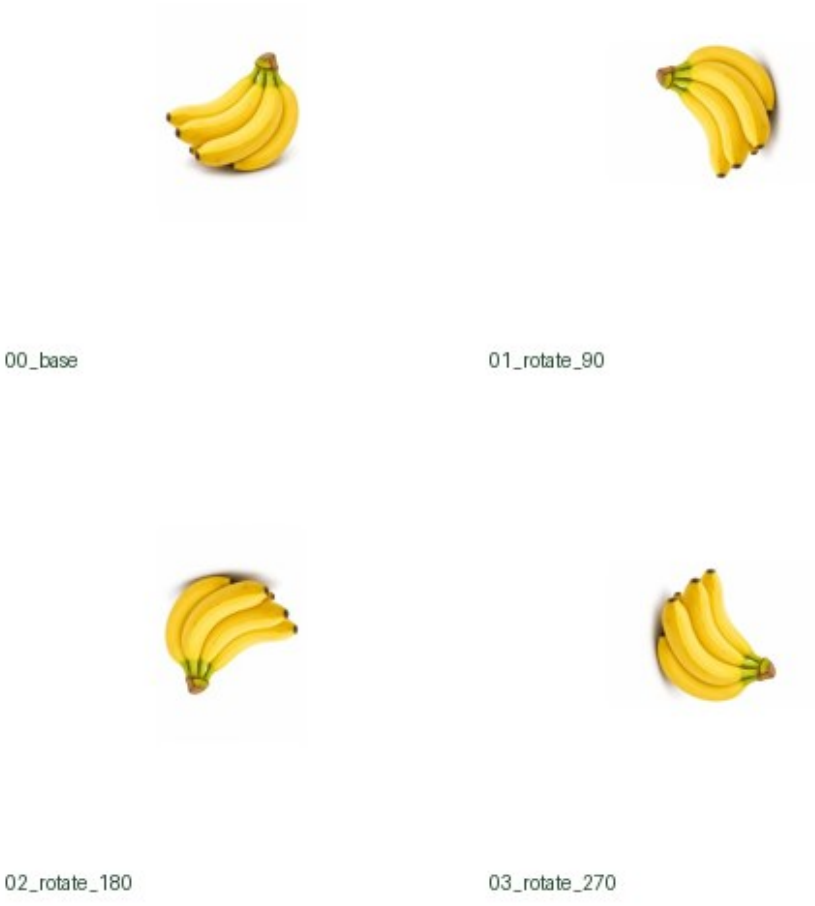
1. IDLE Scale near zero	2. WEIGHT STABLE BLE stream stabilizes	3. CAPTURE Camera grabs tray ROI
4. CLASSIFY Local model predicts SKU	5. CART ADD SKU + net grams + price	6. RE-ARM Wait for scale to return to zero

One weighed transaction should create exactly one cart event. Do not add again on every repeated Bluetooth packet. Use an event ID or a state-machine lock until the scale returns to zero.

2. What is inside the dataset ZIP

The starter ZIP contains 76 fresh-produce classes, 7 optional packaged-product classes, and 336 image files. Each class folder contains 4 images: base, rotate 90, rotate 180, and rotate 270.

Example: the same banana seed image in four orientations



Read this carefully: rotating one studio image does not create a new physical camera angle, new fruit size, a different specimen, or a real shop background. The generated variants help orientation robustness only. They must not be treated as 4 independent captures when splitting train/validation/test.

Source-image quality note: several seed pictures come from compact 256 px POS catalog files and are normalized/upscaled to 640 x 640 for a common model input. Upscaling does not create extra detail. Some catalog compositions may include leaves, flowers, groups, or sliced examples; use real shop-camera captures of complete items to replace or supplement these images before production training.

ZIP structure

AI_Smart_Weighing_POS_Dataset_Starter.zip
images/fresh_produce/<class_id>/00_base.jpg ... 03_rotate_270.jpg
images/optional_packaged_products/<class_id>/...
class_map.csv
image_manifest.csv
dataset_summary.json
README.md
reference/user_scale_camera_reference.jpg

Use the ZIP for image ingestion and the PDF as the instructions/reference document. A PDF alone is not a proper training dataset.

3. Fresh produce class inventory

These are the produce SKUs for which a source catalog/POS image was available in the current conversation files. Class IDs are stable folder names and should be mapped to the POS product master.

Vegetables and leafy greens (52) - part 1

CLASS ID	PRODUCT LABEL	GROUP
adrak	Ginger / Adrak	vegetable
aloo	Potato / Aloo	vegetable
arbi	Taro Root / Arbi	vegetable
bathua	Bathua Greens / Bathua	leafy green
beans	Green Beans / Beans	vegetable
bhindi	Okra / Bhindi	vegetable
black_baingan	Black Eggplant / Black Baingan	vegetable
black_bharta_baingan	Large Black Bharta Eggplant / Black Bharta Baingan	vegetable
broccoli	Broccoli / Broccoli	vegetable
chukandar	Beetroot / Chukandar	vegetable
desi_mirchi	Green Chilli / Desi Mirchi	vegetable
dhaniya	Coriander / Dhaniya	leafy green
gavar_fali	Cluster Beans / Gavar Fali	vegetable
gilki_tori	Sponge Gourd / Gilki Tori	vegetable
gobhi	Cauliflower / Gobhi	vegetable
green_capsicum	Green Capsicum / Green Shimla Mirch	vegetable
hari_lahsun	Green Garlic / Hari Lahsun	vegetable
hari_methi	Fenugreek Greens / Hari Methi	leafy green
kaddu	Pumpkin / Kaddu	vegetable
karela	Bitter Gourd / Karela	vegetable
kundru	Ivy Gourd / Kundru	vegetable
lahsun	Garlic / Lahsun	vegetable
lal_aloo	Red Potato / Lal Aloo	vegetable
lal_baingan	Red Eggplant / Lal Baingan	vegetable
mixed_capsicum	Mixed Colour Capsicum / Lal-Pili Shimla Mirch	vegetable

3. Fresh produce class inventory (continued)

Vegetables and leafy greens - part 2

CLASS ID	PRODUCT LABEL	GROUP
lauki	Bottle Gourd / Lauki	vegetable
mushroom	Button Mushroom / Mushroom	vegetable
matar	Green Peas / Matar	vegetable
moti_bharva_mirchi	Large Stuffed Chilli / Moti Bharva Mirchi	vegetable
muli	Radish / Muli	vegetable
orange_carrot	Orange Carrot / Orange Gajar	vegetable
palak	Spinach / Palak	leafy green
parwal_kachri	Rajasthani Parwal Kachri / Parwal Kachri	vegetable
pudina	Mint / Pudina	leafy green
pyaj	Red Onion / Pyaj	vegetable
safed_pyaj	White Onion / Safed Pyaj	vegetable
salad_hybrid_tamatar	Hybrid Salad Tomato / Salad Hybrid Tamatar	vegetable
sangri_fali	Sangri Pods / Sangri ki Fali	vegetable
sem_fali	Flat/Field Beans / Sem Fali	vegetable
shakarkandi	Sweet Potato / Shakarkandi	vegetable
singhada	Water Chestnut / Singhada	vegetable
tamatar	Tomato / Tamatar	vegetable
tar_kakdi	Long Green Cucumber / Tar Kakdi	vegetable
tinda	Round Gourd / Tinda	vegetable
tori	Ridge Gourd / Tori	vegetable
kheera	Cucumber / Kheera	vegetable
english_kheera	English Cucumber / English Kheera	vegetable
tikhi_barik_mirchi	Thin Hot Green Chilli / Tikhi Barik Mirchi	vegetable
red_carrot	Red Carrot / Lal Gajar	vegetable
bel_patra_leaves	Bael Leaves / Bel Patra	leafy green
chappan_tinda	Chappan Tinda / Chappan Tinda	vegetable
chola_fali	Chola Fali / Chola Fali	vegetable

3. Fresh produce class inventory (continued)

Fruits (24)

CLASS ID	PRODUCT LABEL	GROUP
aavla	Indian Gooseberry / Aavla	fruit
amrood	Guava / Amrood	fruit
anaar	Pomegranate / Anaar	fruit
bael_fruit	Bael Fruit / Bel Fruit	fruit
chikoo	Sapodilla / Chikoo	fruit
desi_orange	Desi Orange / Desi Santra	fruit
green_orange	Green Orange / Green Orange	fruit
jamun	Java Plum / Jamun	fruit
katahal	Jackfruit / Katahal	fruit
kele	Banana / Kele	fruit
kivi	Kiwi / Kivi	fruit
malta	Malta Orange / Malta	fruit
mini_orange	Mini Orange / Mini Santra	fruit
nariyal_pani	Tender Coconut / Nariyal Pani	fruit
papita	Papaya / Papita	fruit
pear	Pear / Nashpati	fruit
persimmon	Persimmon / Persimmon	fruit
pineapple	Pineapple / Pineapple	fruit
plum	Plum / Plum	fruit
ramfal	Ramfal / Ramfal	fruit
sitafal	Custard Apple / Sitafal	fruit
strawberry	Strawberry / Strawberry	fruit
tarbooj	Watermelon / Tarbooj	fruit
nimbu	Lemon / Nimbu	fruit

Optional packaged items (kept separate from the produce model)

CLASS ID	PRODUCT LABEL
cow_milk	Cow Milk / Milk
buffalo_milk	Buffalo Milk / Buffalo Milk
chhach	Buttermilk / Chhach
zahidi_dates	Zahidi Dates / Zahidi Khajoor
baran_iranian_dates	Baran Iranian Dates / Baran Dates
seer_shah_seedless_dates	Seer Shah Seedless Dates / Seedless Dates
jk_selected_dates	J.K. Selected Dates / JK Khajoor

Keep milk, chhach, and packaged dates in a separate model head or product family. Do not mix them into fresh fruit/vegetable training unless those SKUs are actually weighed and recognized by the same camera workflow.

4. Real image capture plan - required before production

The current seed pack is not enough to train a reliable 76-class shop model. Begin a physical capture campaign with the exact camera and tray that will be used in billing.

Suggested starting volume per SKU

Stage	Images per SKU	Purpose
Pilot	150-300 real photos	Check if class is visually separable and camera framing works.
Production expansion	300-500+ real photos	Capture multiple stock batches, days, sizes, and natural appearance changes.
Hard cases	Additional targeted images	Similar varieties, bruises, dust, leaves, partial occlusion, low light, and edge placement.

For each SKU, deliberately capture these variations

- Placement: center, near each edge, diagonally, front/back of tray, partially rotated, upside down where physically plausible.
- Orientation: horizontal, vertical, diagonal, rotated 90/180/270 degrees. For bunches or leafy items, capture several natural arrangements.
- Appearance: small/large specimens, different sizes, natural colour variation, mild blemishes, wet/dry surface, leaves/stems present or absent.
- Quantity: single piece, small group, bunch, and normal retail quantity. Match how the cashier will actually place them.
- Lighting and context: morning/evening, store lights, shadows, and the final tray/camera installation. Keep a portion of photos with realistic hands/handling, but do not let hands hide the product.

Capture rule: one SKU per weigh event. If a customer puts two different products on the scale together, the vision model cannot safely infer two separate weights from one total weight. Ask the cashier to weigh them separately.

Labeling protocol

- Use the exact `class_id` from `class_map.csv`; also store the POS SKU ID, date/time, stock batch, and camera ID.
- Keep the unmodified original photo. Do not label augmented copies as separate real-world observations.
- Split train/validation/test by capture session, day, and stock batch (e.g. 70/15/15). All rotations of one base image must stay in the same split.

5. AI model and real-time inference design

Recommended model path

Start with image classification if one product/SKU is placed on the scale at a time. First crop a fixed region of interest (ROI) around the tray, then classify the product image. Use an object detector only if multiple different product classes may be present in one camera frame and you need separate object boxes. Detector training requires bounding-box labels; the current ZIP does not contain YOLO bounding-box annotations.

Suggested prediction response

Model result	POS action	Safety intent
Top-1 confidence >= 0.92 and clear margin	Candidate for auto-add	Only after thresholds are calibrated on real store test data.
0.75 to 0.92 or close top-1/top-2 scores	Show top-3 choices for cashier confirmation	Prevents silent wrong-product billing.
Below 0.75, unknown class, or poor image	Do not auto-add; ask for recapture/manual choice	Reject low-confidence guesses.

These are starting thresholds, not guaranteed correct values. Tune them using a held-out test set and a confusion matrix. Similar-looking products may need higher thresholds or additional real images.

The 200 ms target

- Set 200 ms as a target for local model inference after the frame is captured and preprocessed, not as an end-to-end promise.
- A cloud API can add network delay; a small on-device model is more suitable for a fast counter workflow.
- Measure capture-to-cart latency at median and p95 on the exact Android/POS device. Also log Bluetooth weight stabilization time.
- Do not optimize for speed by removing low-confidence safeguards. A fast wrong SKU is worse than a brief confirmation.

Required model outputs

`class_id`, `top_1_score`, `top_2_score`, `top_3_predictions`, `unknown_flag`, `frame_timestamp`, `model_latency_ms`, and a unique `weigh_event_id`.

6. Bluetooth scale + POS cart logic

Weight-to-cart calculation

amount = weight_g / 1000 x price_per_kg
Example only: 650 g at Rs 80/kg = 0.650 x 80 = Rs 52.00. The price must come from the POS product master, not from the image model.

Scale event logic

State	Condition	POS behavior
IDLE	Net weight is near zero	Wait for new product.
STABILIZING	Weight exceeds minimum and readings settle	Do not add yet; wait for scale stable flag or stable consecutive readings.
CAPTURE + CLASSIFY	Stable weight event	Capture one ROI frame; run classification.
CART ADD	SKU accepted; weight stable	Add exactly one cart line with SKU, grams, price, amount, and event ID.
WAIT FOR ZERO	Product removed; scale returns to zero	Close event and re-arm for the next item.

Implementation safeguards

- Use the scale hardware stable indicator if available. Otherwise require a tuned stability window (for example, three consecutive readings within a small gram tolerance for around 250-400 ms). Calibrate this to the scale noise and resolution.
- Use a unique `weigh_event_id` and a state lock so repeated BLE packets cannot add the same item twice.
- Tare the empty basket/tray before selling. Never auto-tare while product is on the platform.
- When the scale reports zero after removal, reset the event and allow the next weighed product.
- Provide manual correction: change SKU, edit weight only when authorized, remove line, or recapture photo.
- For per-piece products, use a piece-count mode or manual override; do not blindly apply kg pricing to every SKU.

7. Testing, safety, and rollout checklist

Before enabling auto-add at the counter

- Camera is fixed and the full tray ROI remains in frame.
- Tray/basket, background, lighting, and camera settings match the real training photos.
- Bluetooth net weight is correct, stable, and in grams; tare was tested.
- Every class has real images from multiple days and stock batches.
- Train/validation/test splits are grouped by capture session; no augmented siblings leak across splits.
- Confusion matrix reviewed for similar classes (e.g., different cucumbers, gourds, eggplants, oranges, carrots, and chillies).
- Unknown/other produce class and low-confidence fallback are enabled.
- Each weigh event can add only once; removal-to-zero re-arms the system.
- Price and unit are fetched from the POS product master; vision model never invents price.
- Latency measured end-to-end on actual counter hardware; no network dependency for the critical recognition path if 200 ms is required.

Metrics to monitor

Metric	Why it matters
Per-class precision / recall	Shows which products are often confused or missed.
False auto-add rate	Most important billing safety metric; measure wrong SKU additions per 1,000 weigh events.
Unknown rejection rate	Ensures unfamiliar items are not forced into a wrong SKU.
Median and p95 capture-to-cart latency	Measures real cashier experience, including BLE and camera delays.
Manual correction rate	Highlights weak classes and the next items that need more real photos.

Recommended rollout: shadow mode first (AI predicts but cashier confirms) -> supervised auto-add for high-confidence classes -> wider auto-add only after false-add rate is acceptably low. Keep a manual fallback at all times.

Bottom line

The ZIP gives the AI agent a labeled starter set of the products already represented in the catalog images. The real accuracy will come from adding genuine photographs captured by the camera above your actual weighing scale. Use the PDF as the training brief and the ZIP as the image seed package.

Prepared from the currently available POS/catalog images in this conversation. Some product names are retained as separate classes because they were created as separate POS SKUs; confirm their exact POS SKU mapping in `sku_mapping_template.csv` before training.